Ansible & AWS — Defining Regions

Assignment | Q1: How do you define an AWS region in Ansible? — with example code

Q1 Defining an AWS Region in Ansible

Four common approaches — from quick inline to production-ready

Overview

When Ansible talks to AWS, it needs to know **which region** to create or manage resources in — for example, *us-east-1* (N. Virginia), *ap-south-1* (Mumbai), or *eu-west-1* (Ireland). The region is passed to AWS modules like **amazon.aws.ec2_instance**, **amazon.aws.s3_bucket**, and others. There are four main ways to define it, ranging from a quick inline value all the way to a clean, environment-driven variable setup used in real projects.

1 Inline in the Task (Direct Value)

Simplest approach — hardcode the region directly in the module

The quickest way is to pass the **region** parameter directly inside the task that calls an AWS module. This is fine for quick tests or one-off scripts, but hardcoding a region in every task makes playbooks harder to reuse across different environments.

```
# launch_ec2.yml
---
- name: Launch EC2 instance
hosts: localhost
tasks:
- name: Create EC2 instance in Mumbai
amazon.aws.ec2_instance:
name: my-server
region: ap-south-1 # <-- region defined inline here
instance_type: t2.micro
image_id: ami-0c55b159cbfafe1f0
key_name: my-keypair
state: present
```

▲ *Region hard-coded directly inside the module task*

2 Playbook Variable (vars block)

Define once at the top, reuse across all tasks

A better approach is to define **aws_region** in the *vars* block at the top of the playbook and reference it using **{{ aws_region }}** in every task. Now changing the region means updating one line, not hunting through every task.

```
# launch_ec2.yml
---
- name: Launch EC2 instance
hosts: localhost
vars:
aws_region: ap-south-1 # <-- defined once here
instance_type: t2.micro
ami_id: ami-0c55b159cbfafe1f0
tasks:
- name: Create EC2 instance
amazon.aws.ec2_instance:
name: my-server
region: "{{ aws_region }}" # <-- referenced here
instance_type: "{{ instance_type }}"
image_id: "{{ ami_id }}"
key_name: my-keypair
state: present
```

▲ *Region defined in vars block and referenced with Jinja2 {{ }} syntax*

3 group_vars File (Recommended for Teams)

Store region per environment — dev, staging, prod

For real projects, store variables in **group_vars/** or **host_vars/** files. Ansible automatically loads these based on the inventory group the host belongs to. This means your *dev* group can target *ap-south-1* while *prod* targets *us-east-1* — same playbook, different variables, zero code changes.

```
# group_vars/all.yml (applies to every host)
aws_region: ap-south-1
instance_type: t2.micro
# group_vars/prod.yml (overrides for production group)
aws_region: us-east-1
instance_type: t3.large
# launch_ec2.yml (playbook stays unchanged)
- name: Launch EC2
hosts: localhost
tasks:
- name: Create instance
amazon.aws.ec2_instance:
region: "{{ aws_region }}" # pulled from group_vars automatically
instance_type: "{{ instance_type }}"
state: present
```

▲ *Ansible auto-loads group_vars — no imports needed in the playbook*

4 Runtime Override with -e (Extra Vars)

Pass region at run time — great for CI/CD pipelines

You can override any variable at runtime using the **-e** flag. This is the **highest-precedence** method — it beats everything else, including group_vars. CI/CD pipelines (Jenkins, GitHub Actions) use this to pass the target region as a pipeline parameter without ever modifying a file.

```
# Run playbook and pass region at the command line
ansible-playbook launch_ec2.yml -e "aws_region=eu-west-1"
# Pass multiple overrides at once
ansible-playbook launch_ec2.yml \
-e "aws_region=us-west-2" \
-e "instance_type=t3.medium"
# Inside the playbook, the variable is used the same way
# region: "{{ aws_region }}" <-- no change needed in the YAML
```

▲ *-e flag overrides have the highest precedence — always win*

Quick Comparison

Method	Where Defined	Best For	Precedence
Inline value	Inside the task	Quick tests / one-offs	Low
vars block	Top of playbook	Single-playbook projects	Medium
group_vars file	Separate YAML file	Team projects, multi-environment	Medium-High
-e flag (runtime)	Command line	CI/CD pipelines, overrides	**Highest ★**

aws_region param	**Best Practice**	**Syntax**
Key parameter in all AWS modules	*Use group_vars for teams, -e for CI/CD*	*region: "{{ aws_region }}"*